

Predictive Routing Replay for MoE-based LLM Reinforcement Learning

Anonymous Authors¹

CHECK 1. Distributions p_θ and q SHOULD BE ASSIGNED
A NAME. 2. Check terminology consistency! (online routing
distribution) 3. Check symbol consistency!

Abstract

Mixture of Experts (MoE) Large Language Models (LLMs) achieve strong performance at scale. However, reinforcement learning (RL) on MoE-based LLMs often suffers from training instability. A root cause is *router drift*, i.e., expert activations can change drastically across model updates and differ between disaggregated inference and training phases, causing large training–inference mismatch and unstable importance sampling weights in PPO-style RL algorithms. Existing routing replay (R2) method seeks to mitigate this issue by freezing routing per reasoning trajectory, but it ignores how the router evolves under off-policy updates and causes router-level *policy staleness*. To address this limitation, we propose **Predictive Routing Replay (PR2)**, which augments each router with a lightweight bias predictor that learns to predict short-horizon router evolution to facilitate router evolution. During inference, we use the predicted routing distribution to apply top- k routing, enabling gradients to reach experts that are likely to become active after updates. During training, we replay the same predicted routing path to retain consistency for stable importance estimation. Both theoretical analysis and extensive experimental results have demonstrated that PR2 reduces routing-induced mismatch, improves RL stability, and yields stronger performance across various reasoning benchmarks.

1. Introduction

Large language models (LLMs) have demonstrated strong reasoning capabilities when scaled through increased model

parameters, data, and training compute (OpenAI, 2024; Liu et al., 2024a; Team, 2025a). Mixture of Experts (MoE) models have recently emerged as a promising LLM architecture, enabling training compute to scale sublinearly with model size (Jiang et al., 2024; Zhu et al., 2024; Liu et al., 2024a; Yang et al., 2025; Agarwal et al., 2025; Team et al., 2025a; Team, 2026). This efficiency is achieved by activating a subset of expert networks within the large model, where the activated parameters typically account for approximately 10% of the total model parameters (Liu et al., 2024a; Team, 2026). This conditional computation paradigm has enabled trillion-parameter-scale models with manageable inference costs and has become a core design choice in modern large-scale LLMs (Team et al., 2025a;b).

Beyond pretraining, reinforcement learning (RL) emerges as a central mechanism for optimizing pretrained LLMs toward improved reasoning and alignment with human preferences (Ouyang et al., 2022; Rafailov et al., 2023; Shao et al., 2024; Chen et al., 2025; Team, 2025b; Liu et al., 2025). Algorithms such as Proximal Policy Optimization (PPO) (Schulman et al., 2017) and its variants (Shao et al., 2024; Guo et al., 2025; Yu et al., 2025) are now widely used to improve multi-step reasoning, long-horizon decision making, and tool use tasks (Chen et al., 2022; Yao et al., 2023). However, applying RL to MoE-based LLMs introduces a unique and underexplored set of stability challenges that do not arise in dense models (Yao et al., 2025).

A key difficulty for conducting stable RL on MoE-based LLMs stems from the presence of learned routers that dynamically assign tokens to experts (Yao et al., 2025; Zheng et al., 2025a; Kim et al., 2025). During RL training, rollout data are often generated by a frozen or lagged inference engine, while policy optimization is performed on a continuously updated training model. As a result, expert assignments for the same token can differ substantially between inference-time and training-time forward passes. This *router drift* induces severe training–inference mismatch: policy gradients for the same token are computed under inconsistent routing paths. This discrepancy manifests as a large variance in the importance ratio under PPO-style objectives, thereby undermining the stability of policy optimization across numerous on-policy RL algorithms (Schulman et al., 2017; Shao et al., 2024; Yu et al., 2025).

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

To overcome the training–inference mismatch, a common solution is Routing Replay (R2), which caches routing paths during inference and replays them during training (Zheng et al., 2024; Ma et al., 2025). While effective at restoring routing consistency, R2 introduces its own limitations. By fixing routing paths, it prevents gradients from reaching experts that would become active after subsequent policy updates. Over time, this induces router-level *policy staleness*, as newly activated experts under the updated routing policy are excluded from gradient updates. Theoretically, this results in biased optimization, where the gradient estimator is computed under a routing distribution that deviates from the evolving target policy.

In this work, we revisit routing replay from a principled perspective. We formalize routing replay as replacing the true target routing distribution with a degenerate replay measure, and show that this induces a gradient bias bounded by the divergence between the current routing distribution and the replayed one. This view reveals:

Stable importance estimation favors frozen routing, while effective learning requires routing distributions that track policy evolution.

Motivated by this perspective, we propose **Predictive Routing Replay (PR2)**, which augments routing replay with a routing path prediction scheme. PR2 augments the router in each MoE layer with a lightweight bias predictor that anticipates *router drift*. During inference, the predictor outputs a learned logit bias to adjust the routing distribution. We then perform top- k routing under the biased logits to obtain a predicted expert set, which is cached alongside trajectories for subsequent updates. During training, we replay the cached top- k expert sets while disabling the predictor, keeping the routing path fixed to stabilize PPO-style importance ratio yet allowing gradient flow to experts that are likely to become active after policy updates (Figure 1). We train the bias predictor with a KL-divergence objective derived from an upper bound on routing-induced gradient bias, along with an efficient training scheme and a dedicated learning-rate multiplier for fast adaptation.

PR2 integrates seamlessly into existing MoE RL framework, e.g. Verl (Sheng et al., 2025) and Slime (Zhu et al., 2025), without modifying the policy optimization objective. It adds negligible additional overhead due to the bias predictor’s small footprint, whose computation can be further overlapped with other computation or communication during RL. Empirically, PR2 substantially reduces training–inference mismatch, stabilizes RL training, and improves performance over strong baselines (Guo et al., 2025; Yu et al., 2025; Zheng et al., 2025b) across reasoning tasks. Notably, GRPO with PR2 achieves 40.31% accuracy on AIME24 (Zhang & Math-AI) using the Qwen3-MoE-30B-

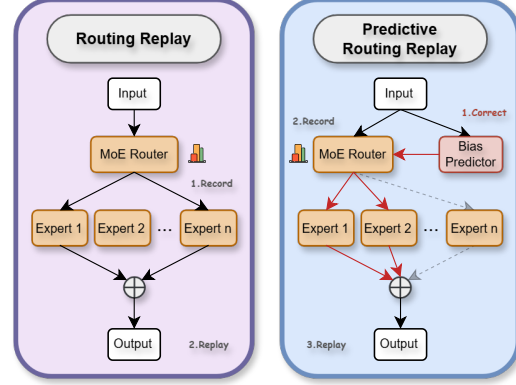


Figure 1. Architecture of Predictive Routing Replay (PR2). PR2 adds a lightweight bias predictor to adjust the router logits during rollout, selects the top- k experts under the biased logits, and caches the resulting expert sets for replay during training.

A3B base model (Yang et al., 2025), outperforming R2 and GSPO baselines by 9.90% and 15.21%, respectively.

Our main contributions are summarized as follows:

- We formalize *router staleness* as a key source of degeneration in routing replay for MoE-based LLM reinforcement learning, and derive a divergence-based upper bound to show that it can lead to biased gradient estimation in PPO-style training.
- We propose **Predictive Routing Replay (PR2)**, which predicts short-horizon router evolution via lightweight bias predictors trained with a KL objective motivated by the above bound, enabling inference-time routing to anticipate future expert activation. **DDZ can neglect the KL loss part**
- We demonstrate that PR2 significantly reduces training–inference mismatch, improves RL stability, and yields stronger performance across several reasoning tasks, establishing a practical and theoretically motivated alternative to routing replay.

2. Related Work

Mixture of Experts. The Mixture of Experts (MoE) layer scales model capacity through conditional computation, where a learned router selects a subset of expert networks per token to reduce FLOPs while retaining a large number of parameters (Shazeer et al., 2017; Lepikhin et al., 2020; Fedus et al., 2022; Zhu et al., 2024; Jiang et al., 2024; Yang et al., 2025; Team et al., 2025a). A large body of work studies the routing strategy (Lewis et al., 2021; Huang et al., 2024), load balancing (Shazeer et al., 2017; Fedus et al., 2022; Liu et al., 2024a), and gradient estimation (Kool et al., 2021; Liu et al., 2023; 2024b) in MoE training. Analysis also highlights that routing decisions can be fragile (Dai et al.,

2022; Zoph et al., 2022), which leads to biased gradient flow and unstable optimization during training (Kim et al., 2025). These findings motivate treating router evolution as an explicit modeling target, especially in disaggregated training–inference settings (Yao et al., 2025).

Reinforcement Learning in LLMs. Reinforcement learning (RL) is widely used to align LLMs for improved reasoning and instruction following, with PPO-style algorithms and their variants serving as the dominant practical backbone (Ouyang et al., 2022; Shao et al., 2024; Guo et al., 2025; Yu et al., 2025; Zhao et al., 2026). Large-scale LLM RL is often implemented in disaggregated pipelines, where rollouts and training are lagged, thereby introducing off-policy effects (Mnih et al., 2016; Espeholt et al., 2018). For MoE-based LLMs, this mismatch is exacerbated by *router drift*: the same token can be assigned to different experts during inference-time and training-time forward passes. This routing-induced discrepancy can amplify variance in importance ratios and destabilize policy optimization, harming the RL performance (Yao et al., 2025).

Stabilizing RL in MoE. To overcome *router drift*, a common approach is Routing Replay (R2) (Zheng et al., 2024), which caches inference-time expert sets and reuses them during training to ensure routing consistency. Rollout Routing Replay (R3) (Ma et al., 2025) further aligns routing consistency by replaying rollout-time routing decisions, substantially reducing training–inference mismatch. Beyond routing replay, GSPO (Zheng et al., 2025b) defines the importance ratio based on sequence likelihood and performs sequence-level clipping, avoiding token-level importance discrepancies induced by *router drift*. Other methods improve stability by modifying the clipping behavior (Gao et al., 2025) or directly adjusting importance ratios to reduce sensitivity to off-policy errors (Zhang et al., 2025; Team, 2025b). In contrast, our work revisits routing replay by analyzing bias induced by replaying a degenerate routing measure, and propose to predict short-horizon router evolution to improve gradient estimation.

3. Preliminaries

We study the reinforcement learning of an autoregressive MoE LLM in a disaggregated pipeline, where rollouts are generated by an inference engine and policy updates are performed in a separate training engine.

3.1. Disaggregated Reinforcement Learning

Let $\theta, \theta_{\text{old}}$ denote the model parameters for training and used to generate rollouts, respectively. Given a prompt x , a behavior policy $\mu_{\theta_{\text{old}}}$ samples a completion $y = (y_1, \dots, y_T)$ token-by-token. At step t , the state is $s_t = (x, y_{<t})$ and the

action is $a_t = y_t$, where a reward $R(x, y)$ is assigned after generation. Let π_θ denote the current training policy. In disaggregated RL, policy optimization is commonly written as an importance-weighted objective:

$$J(\theta) = \mathbb{E}_{x, y \sim \mu_{\theta_{\text{old}}}} [w_\theta(x, y) R(x, y)]. \quad (1)$$

Methods like PPO (Ouyang et al., 2022) and GRPO (Shao et al., 2024; Guo et al., 2025) typically factorize importance weights w_θ into token-level ratios,

$$w_\theta(x, y) = \prod_{t=1}^T r_t(\theta), \quad r_t(\theta) = \frac{\pi_\theta(y_t | x, y_{<t})}{\mu_{\theta_{\text{old}}}(y_t | x, y_{<t})}. \quad (2)$$

To separate train–infer mismatch from parameter staleness, we insert $\pi_{\theta_{\text{old}}}$ following Zheng et al. (2025a):

$$r_t(\theta) = \underbrace{\frac{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})}{\mu_{\theta_{\text{old}}}(y_t | x, y_{<t})}}_{\text{① train–infer discrepancy}} \cdot \underbrace{\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})}}_{\text{② policy staleness}}. \quad (3)$$

We focus on PPO-style policy optimization, where training is driven by importance ratios in Eq. (2). We summarize the detailed objectives in Appendix A.

3.2. Mixture of Experts

We consider an MoE LLM with L MoE layers and N experts per layer. At token t and layer l , the router produces logits and routing probabilities over experts, and selects a top- k expert index set $\mathcal{I}_t^{(l)} \subseteq \{1, \dots, N\}$. We denote the layer-wise routing path at step t as

$$\mathcal{R}_t := \{\mathcal{I}_t^{(1)}, \dots, \mathcal{I}_t^{(L)}\}. \quad (4)$$

Let $\rho_{\theta,t}(\mathcal{R}_t | x, y_{<t})$ denote the online routing distribution induced by the routers of π_θ at token t (possibly degenerate under deterministic top- k selection). Conditioned on a routing path, the token distribution is $\pi_\theta(a_t | x, y_{<t}, \mathcal{R}_t)$, and the marginal token distribution satisfies

$$\pi_\theta(y_t | x, y_{<t}) = \mathbb{E}_{\mathcal{R}_t \sim \rho_{\theta,t}} [\pi_\theta(y_t | x, y_{<t}, \mathcal{R}_t)]. \quad (5)$$

Router drift. In disaggregated pipelines, the same prefix $(x, y_{<t})$ may induce different routing paths across phases. Let $\mathcal{R}_{\text{old},t}^\mu$ denote the rollout-time route chosen by the behavior policy $\mu_{\theta_{\text{old}}}$ (inference engine), and let $\mathcal{R}_{\theta,t}^\pi$ denote the update-time route induced by the current training policy π_θ (training engine). We denote their routing distributions by $\rho_{\text{old},t}^\mu(\cdot | x, y_{<t})$ and $\rho_{\theta,t}^\pi(\cdot | x, y_{<t})$, respectively. Router drift refers to the discrepancy $\mathcal{R}_{\text{old},t}^\mu \neq \mathcal{R}_{\theta,t}^\pi$ (or $\rho_{\theta,t}^\pi \neq \rho_{\text{old},t}^\mu$), which yields inconsistent expert activation for the same token and can amplify the variance of importance ratios in PPO-style optimization (Zheng et al., 2025a; Yao et al., 2025).

3.3. Routing Replay

Routing replay enforces routing consistency by replacing the online routing distribution $\rho_{\theta,t}(\cdot | x, y_{<t})$ with a cached replay measure supported on a single route:

$$q_t(\mathcal{R}_t | x, y_{<t}) = \delta_{\tilde{\mathcal{R}}_t}(\mathcal{R}_t), \quad (6)$$

$$\pi_{\theta}^q(y_t | x, y_{<t}) := \pi_{\theta}(y_t | x, y_{<t}, \tilde{\mathcal{R}}_t),$$

where $\tilde{\mathcal{R}}_t$ is a cached routing path. R2 sets $\tilde{\mathcal{R}}_t = \mathcal{R}_{\text{old},t}^{\pi}$ by replaying training-engine routing from the previous policy snapshot (Zheng et al., 2024), while R3 sets $\tilde{\mathcal{R}}_t = \mathcal{R}_{\text{old},t}^{\mu}$ by replaying inference-engine routing (Ma et al., 2025). Both stabilize importance estimation by fixing $\mathcal{R}_t$ during gradient evaluation, but may prevent gradients from reaching experts that would become active under subsequent router updates.

4. Predictive Routing Replay

Building on routing replay in Section 3, we propose **Predictive Routing Replay (PR2)** for MoE RL under disaggregated rollout/training. The key tension is: *frozen replay routes stabilize importance-ratio estimation, but they become increasingly stale as routers evolve, which induces routing mismatch and biased (or misdirected) updates*. PR2 keeps replay deterministic for training-time consistency, while reducing staleness by *predicting short-horizon router evolution* during rollout.

4.1. PR2 Overview

PR2 augments the router in each MoE layer with a lightweight predictor head. Crucially, the predictor is used *only* in the rollout engine to construct a *predicted* top- k expert set; training disables the predictor and replays the cached expert sets.

(1) Rollout / route caching. During rollout with the behavior policy $\mu_{\theta_{\text{old}}}$, PR2 adds a learned logit bias to the router logits, performs top- k routing under the biased logits, and caches the resulting expert sets as replay routes. To keep buffers feasible under long contexts, we cache router features (e.g., $(h_{\text{old}}, p_{\text{old}})$) only on a *subsampled* set of token positions along the sequence; predictor losses are evaluated on this subset.

(2) Training / deterministic replay. During policy updates under π_{θ} , PR2 replays the cached top- k expert sets and disables the predictor, ensuring the routing path is fixed within each replayed trajectory, as in prior routing replay methods.

(3) Predictor update. PR2 updates predictor parameters by aligning the predicted routing distribution to the current routing distribution using a KL loss (Eq. (15)). This loss

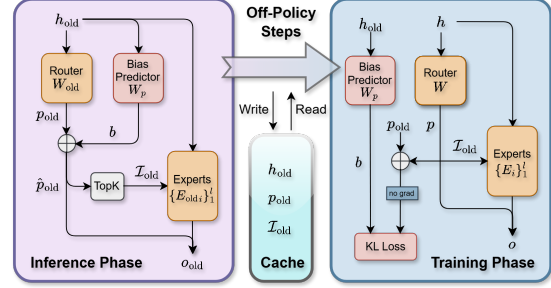


Figure 2. Detailed PR2 workflow. **Rollout under $\mu_{\theta_{\text{old}}}$:** PR2 predicts router evolution by adding a learned logit bias, applies top- k routing, and caches predicted expert sets (and subsampled router features for predictor learning). **Training under π_{θ} :** the cached expert sets are replayed for route consistency; the predictor is updated with post-routing matching so replay routes remain close to the evolving router.

is a smooth proxy for routing mismatch and empirically improves top- k agreement and reduces replay divergence (Section 5).

4.2. Replay Object and Predictive Route Construction

Replay object. PR2 replays the *top- k expert index set* at each MoE layer. For token t and layer l , the replayed set is denoted by $\hat{\mathcal{I}}_{\text{old},t}^{(l)} \subseteq \{1, \dots, N\}$, and the layer-wise replay path is $\hat{\mathcal{R}}_{\text{old},t} := \{\hat{\mathcal{I}}_{\text{old},t}^{(l)}\}_{l=1}^L$. While the replayed object is discrete (a set), we will also use a soft routing distribution $\hat{q}_t^{(l)}$ (defined below) as a differentiable proxy for learning the predictor.

Predictive routing distribution. At rollout time, the router in layer l produces logits

$$p_{\text{old},t}^{(l)} = h_{\text{old},t}^{(l)} W_{\text{old}}^{(l)}, \quad (7)$$

and the predictor outputs a bias

$$b_t^{(l)} = h_{\text{old},t}^{(l)} W_p^{(l)}. \quad (8)$$

We form the predicted routing distribution

$$\hat{q}_t^{(l)} = \text{Softmax}(p_{\text{old},t}^{(l)} + b_t^{(l)}), \quad (9)$$

and take the predicted replay set

$$\hat{\mathcal{I}}_{\text{old},t}^{(l)} = \text{TopK}(\hat{q}_t^{(l)}, k), \quad \hat{\mathcal{R}}_{\text{old},t} = \{\hat{\mathcal{I}}_{\text{old},t}^{(l)}\}_{l=1}^L. \quad (10)$$

Rollout phase. PR2 generates tokens under the predicted route $\hat{\mathcal{R}}_{\text{old},t}$ and caches the predicted expert sets for replay. Additionally, PR2 caches router features $(h_{\text{old},t}, p_{\text{old},t})$ only on a subsampled set of token positions to control memory;

this feature buffer is used solely for predictor updates. At layer level, the rollout MoE output is

$$o_{\theta_{\text{old}},t}^{(l)} = \sum_{j \in \hat{\mathcal{I}}_{\text{old},t}^{(l)}} \hat{q}_{t,j}^{(l)} E_{\text{old},j}^{(l)}(h_{\text{old},t}^{(l)}). \quad (11)$$

Training phase. During updates under π_θ , PR2 disables the predictor and replays the cached route $\mathcal{R}_{\text{old},t}$. We compute current routing probabilities

$$\rho_{\theta,t}^{(l)} = \text{Softmax}(p_t^{(l)}), \quad p_t^{(l)} = h_t^{(l)} W^{(l)}, \quad (12)$$

and the training MoE output uses current probabilities on the replayed expert set:

$$\rho_{\theta,t}^{(l)} = \sum_{j \in \hat{\mathcal{I}}_{\text{old},t}^{(l)}} \rho_{\theta,t,j}^{(l)} E_j^{(l)}(h_t^{(l)}). \quad (13)$$

This keeps the *expert support* fixed (replay consistency), while still allowing gradients to reflect the current router’s preference within that support.

4.3. From Replay Mismatch to a Proxy Objective

At token t , let $s_t = (x, y_{<t})$ and let $\mathcal{R}_t$ denote the layer-wise route defined in Eq. (4). Recall from Section 3 that the online route is induced by $\rho_{\theta,t}(\cdot | s_t)$, while routing replay evaluates gradients under a replay measure $q_t(\cdot | s_t)$ (often a Dirac measure on a cached route; Eq. (6)). The ideal MoE policy-gradient contribution is

$$g_t^*(\theta) := \mathbb{E}_{\mathcal{R}_t \sim \rho_{\theta,t}(\cdot | s_t)} [A_t \nabla_\theta \log \pi_\theta(y_t | s_t, \mathcal{R}_t)],$$

whereas replay computes

$$g_t(q_t) := \mathbb{E}_{\mathcal{R}_t \sim q_t(\cdot | s_t)} [A_t \nabla_\theta \log \pi_\theta(y_t | s_t, \mathcal{R}_t)].$$

Using total variation (TV) duality,

$$\|g_t^*(\theta) - g_t(q_t)\| \leq 2M_t D_{\text{TV}}(\rho_{\theta,t}(\cdot | s_t), q_t(\cdot | s_t)), \quad (14)$$

$$M_t \geq \|A_t \nabla_\theta \log \pi_\theta(y_t | s_t, \mathcal{R}_t)\|.$$

Eq. (14) makes the design target explicit: we should keep replay deterministic for stable optimization, but also keep replay routes close to the evolving routing distribution $\rho_{\theta,t}$.

A smooth relaxation. Directly optimizing TV against a deterministic replay measure (Dirac on a top- k set) is non-differentiable. PR2 therefore learns a *soft* predicted routing distribution $\hat{q}_t^{(l)}$ and uses $D_{\text{KL}}(\rho_{\theta,t}^{(l)} \| \hat{q}_t^{(l)})$ as a proxy objective. By Pinsker’s inequality, aligning $\hat{q}_t^{(l)}$ to $\rho_{\theta,t}^{(l)}$ also controls TV between the corresponding categorical distributions. Empirically, this proxy improves the agreement between the predicted top- k set and the current top- k set (Section 5).

4.4. Training Objectives and Optimization

Stop-gradient notation. We use $\langle \cdot \rangle$ to denote stop-gradient (detach) targets.

Auxiliary predictor losses. We consider two objectives for predicting short-horizon router change between θ_{old} and θ .

(1) Delta matching.

$$\mathcal{L}_{\text{PR2}}^\Delta = \sum_{l=1}^L \mathbb{E}_t [D_{\text{KL}}(\text{Softmax}(\langle \Delta p_t^{(l)} \rangle) \| \text{Softmax}(b_t^{(l)}))],$$

$$\text{with } \Delta p_t^{(l)} = p_t^{(l)} - p_{\text{old},t}^{(l)}.$$

(2) Post-routing matching.

$$\mathcal{L}_{\text{PR2}}^{(\text{post})} = \sum_{l=1}^L \mathbb{E}_t [D_{\text{KL}}(\langle \rho_{\theta,t}^{(l)} \rangle \| \hat{q}_t^{(l)})]. \quad (15)$$

We use $\mathcal{L}_{\text{PR2}}^{(\text{post})}$ in the main algorithm because it is well-conditioned and directly aligns predicted and current routing distributions.

Proposition 4.1 (Post-routing matching controls a distributional replay-mismatch proxy). *Assume $\|A_t \nabla_\theta \log \pi_\theta(y_t | s_t, \mathcal{R})\| \leq M$ for all routes $\mathcal{R}$ and tokens t . If routing distributions factorize across MoE layers, then for the relaxed replay measure that samples routes from $\hat{q}_t$,*

$$\mathbb{E}_t [\|g_t^*(\theta) - g_t(\hat{q}_t)\|] \leq M \sqrt{2 \mathcal{L}_{\text{PR2}}^{(\text{post})}}. \quad (16)$$

The full proof is deferred to Appendix C. In PR2, we replay the deterministic top- k sets $\hat{\mathcal{I}}_{\text{old},t}^{(l)} = \text{TopK}(\hat{q}_t^{(l)}, k)$; thus Proposition 4.1 should be viewed as an analysis of a smooth relaxation that motivates the KL objective.

Two-phase overlapped optimization. In each rollout batch reused for multiple mini-steps, PR2 uses the following schedule: **(i)** At mini-step 1 (near on-policy), predictor update is skipped and cached routes are consumed directly. **(ii)** From mini-step 2 onward, PR2 updates W_p via Eq. (15) using stop-gradient targets, evaluated on the subsampled tokens whose router features were cached. **(iii)** Gradients from the auxiliary loss update predictor parameters only; backbone policy optimization remains unchanged.

Learning-rate scaling. Predictors are zero-initialized so PR2 starts from the original router behavior. To avoid slow warm-up, predictor parameters use $\eta_p = \alpha \eta$ with a dedicated multiplier α .

4.5. Comparison with R2 and R3

Table 1 summarizes which routing paths are used across engines/phases. R2/R3 improve stability by replaying stale

Computation	GRPO	R2	R3	PR2
Rollout generation ($\mu_{\theta_{\text{old}}}$)	$\mathcal{R}_{\text{old}}^{\mu}$	$\mathcal{R}_{\text{old}}^{\mu}$	$\mathcal{R}_{\text{old}}^{\mu}$	$\hat{\mathcal{R}}_{\text{old}}$
Old log-prob ($\pi_{\theta_{\text{old}}}$)	$\mathcal{R}_{\text{old}}^{\pi}$	$\mathcal{R}_{\text{old}}^{\pi}$	$\mathcal{R}_{\text{old}}^{\pi}$	$\hat{\mathcal{R}}_{\text{old}}$
Training/backprop (π_{θ})	$\mathcal{R}^{\pi}$	$\mathcal{R}_{\text{old}}^{\pi}$	$\mathcal{R}_{\text{old}}^{\pi}$	$\hat{\mathcal{R}}_{\text{old}}$
Training stability	✗	✓	✓	✓
Gradient to new experts	✓	✗	✗	✓
Mitigates router staleness	✓	✗	✗	✓

Table 1. Routing-path comparison across methods. PR2 preserves deterministic replay for stable optimization while replacing stale replay paths with predicted replay paths.

routes; PR2 keeps deterministic replay but replaces stale replay paths with predicted ones that better follow router evolution.

5. Experiments

We evaluate PR2 on MoE-based LLM reinforcement learning under disaggregated rollout/training. Our experiments target three questions: **RQ1 (performance)**: does PR2 improve reasoning accuracy under different off-policy strengths? **RQ2 (stability)**: does PR2 stabilize PPO-style optimization dynamics? **RQ3 (mechanism)**: does PR2 better track router evolution and reduce replay mismatch?

5.1. Experimental Setup

Model and training data. We implement PR2 in Verl (Sheng et al., 2025) and train Qwen3-MoE-30B-A3B Base (Yang et al., 2025) on DAPO-17K (Yu et al., 2025).

Baselines and protocol. We compare against GRPO (Shao et al., 2024), GSPO (Zheng et al., 2025b), R2 (Zheng et al., 2024), and R3 (Ma et al., 2025). GRPO follows DAPO Clip-Higher without KL regularization. GSPO uses $\epsilon_{\text{low}} = 3 \times 10^{-4}$, $\epsilon_{\text{high}} = 4 \times 10^{-4}$, and KL coefficient 10^{-3} . We adopt SGLang oversampling ratio 0.1 (Zheng et al., 2024) and train one epoch per setting. Full hyperparameters are given in Appendix E.

Metrics. We report Avg@ n accuracy: Avg@32 on AIME24/AIME25 (Zhang & Math-AI), Avg@16 on HMMT25 (Balunović et al., 2025) and AMC23 (Mathematical Association of America, 2026), and Avg@4 on MATH500 (Hendrycks et al., 2021). Validation is evaluated every 10 training steps.

Off-policy strength. We define $\kappa = \mathcal{B}_{\text{global}}/\mathcal{B}_{\text{update}}$ and denote settings as off- κ . Larger κ means stronger rollout-batch reuse and typically larger router drift.

Overhead. PR2 adds one lightweight predictor head per MoE router (a linear projection producing a logit bias), which is negligible relative to expert parameters. Runtime overhead is limited to an extra router-side projection during rollout. To control memory under long contexts, we cache full predicted top- k expert sets for replay, but cache router features used for predictor learning only on a subsampled set of token positions (Section 4 and Appendix E).

5.2. Main Results

RQ1: PR2 improves accuracy over complete baselines. Table 2 summarizes results for off-2/off-4/off-8. Among methods with complete metrics in all three settings (GRPO, GSPO, R2, PR2), PR2 achieves the best average accuracy in every setting: 55.59 (off-2), 54.27 (off-4), and 50.85 (off-8). Compared with R2, PR2 improves by +8.13, +10.49, and +6.87 points, respectively; compared with GSPO, improvements are +9.38, +7.73, and +7.05 points. These gains are most visible on hard reasoning benchmarks (AIME24/AIME25/AMC23), suggesting that predictive replay is especially helpful when policy updates frequently alter expert preference.

R3 achieves strong off-2/off-4 scores in our reproduction. However, one off-8 metric (AMC23) is unavailable in the corresponding run log, so we avoid full three-setting aggregate ranking that relies on complete off-2/off-4/off-8 entries.

RQ2: PR2 improves optimization stability. Figure 3 reveals three consistent trends. First, vanilla GRPO exhibits large policy-gradient loss spikes and abrupt reward drops, indicating unstable ratio dynamics under off-policy reuse. Second, R2 reduces the worst spikes but still shows late-stage oscillation in clipping rates and entropy. Third, PR2 keeps both low/high clipping rates near zero for most of training and yields smoother reward growth with slower entropy collapse. This pattern is consistent with the mechanism in Section 4: predicted deterministic replay reduces routing-induced ratio outliers while preserving replay consistency.

5.3. Ablation Study

Predictive objective: post-routing matching is more reliable. We compare $\mathcal{L}^{\Delta}$ and $\mathcal{L}^{(\text{post})}$. Both losses improve over no predictor, but they differ in optimization behavior. Delta matching supervises logit residuals, which is sensitive to raw logit scale and tends to be noisier. Post-routing matching directly aligns categorical routing distributions, producing smoother auxiliary loss curves and better validation stability. This supports using Eq. (15) as the default objective.

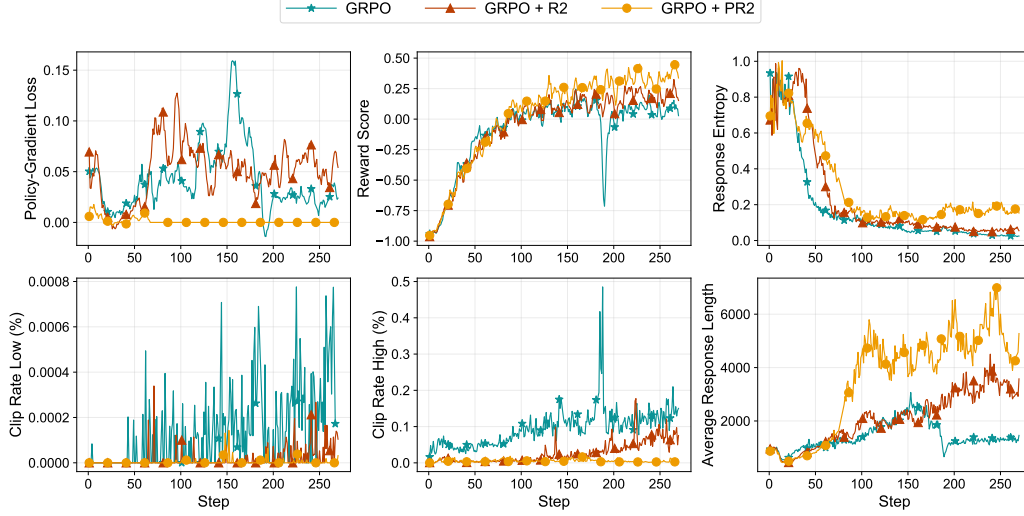


Figure 3. Training dynamics in off-2. We report policy-gradient loss, reward, entropy, low/high clipping rates of token-level importance ratios, and response length for GRPO, GRPO+R2, and GRPO+PR2. PR2 maintains smoother loss/reward trajectories and lower clipping volatility.

Policy	Method	AIME24 (Avg@32)	AIME25 (Avg@32)	HMMT25 (Avg@16)	AMC23 (Avg@16)	MATH500 (Avg@4)	Average
Off-2	GRPO	31.04	24.68	74.06	9.16	84.25	44.64
	GSPO	30.42	23.54	77.18	11.88	86.60	45.92
	GRPO + R2	35.73	25.73	77.50	11.25	85.65	47.17
	GRPO + R3	50.10	33.75	89.69	19.58	89.25	56.47
	GRPO + PR2	47.71	32.81	87.50	19.17	90.75	55.59
Off-4	GRPO	25.42	17.71	70.93	8.33	84.70	41.42
	GSPO	32.50	22.08	79.38	12.08	86.65	46.54
	GRPO + R2	28.13	21.25	73.59	10.00	85.95	43.78
	GRPO + R3	49.90	33.95	89.21	20.41	89.35	56.56
	GRPO + PR2	47.40	31.67	85.47	20.42	86.40	54.27
Off-8	GRPO	25.00	15.93	72.81	7.50	85.40	41.33
	GSPO	30.93	22.18	72.03	7.50	86.35	43.80
	GRPO + R2	28.02	21.77	76.88	8.30	84.95	43.98
	GRPO + R3	45.42	30.83	83.75	—	89.20	—
	GRPO + PR2	40.31	28.54	83.13	15.63	86.65	50.85

Table 2. Downstream reasoning accuracy on Qwen3-MoE-30B-A3B Base across off-policy strengths. Metrics are Avg@ n on five benchmarks. “—” denotes an unavailable metric in the corresponding run log.

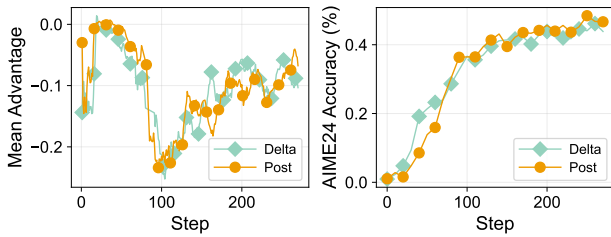


Figure 4. Ablation on predictive objective in off-2. Left: auxiliary loss curves for delta matching vs post-routing matching. Right: AIME24 validation accuracy. Post-routing matching yields more stable optimization and stronger validation behavior.

Learning-rate multiplier α . We sweep the predictor learning-rate multiplier in off-2. A small α underfits predictor updates, while an overly large α destabilizes delta matching. Post-routing matching remains comparatively robust across a wider range, indicating better conditioning

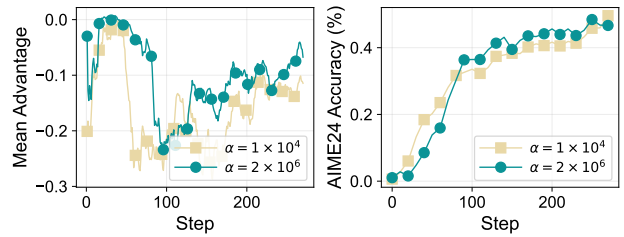


Figure 5. Ablation on predictor learning-rate multiplier α (off-2). Left: auxiliary-loss sensitivity. Right: validation accuracy. Post-routing matching is less sensitive to α than delta matching.

for practical tuning.

5.4. Analysis

RQ3(a): PR2 better tracks evolving routes. Figure 6 (left) reports top- k agreement between the replayed route

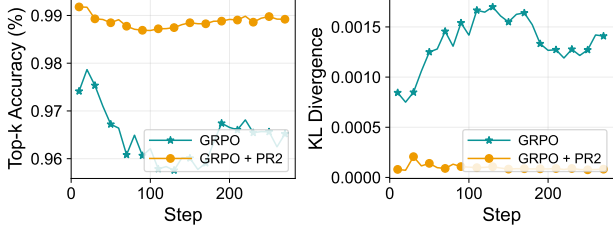


Figure 6. Mechanism analysis in off-2. Left: top- k agreement between replayed/predicted routing and current routing (higher is better). Right: per-layer replay-vs-current routing divergence proxy (lower is better). PR2 increases agreement and decreases divergence.

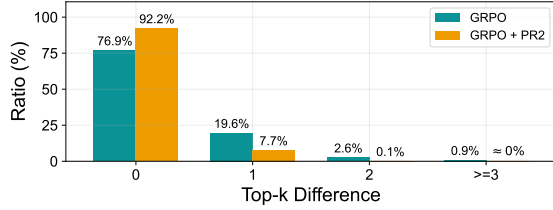


Figure 7. Distribution of top- k routing deviations. X-axis: number of differing experts between replayed and current top- k sets. Y-axis: token fraction. PR2 concentrates mass on smaller deviations.

and the current route. For layer l , let $I_{\text{replay},t}^{(l)}$ be the replayed top- k set and let $I_{\text{cur},t}^{(l)} = \text{TopK}(\rho_{\theta,t}^{(l)}, k)$. We define agreement as $|I_{\text{replay},t}^{(l)} \cap I_{\text{cur},t}^{(l)}|/k$ and average over tokens/layers. Higher agreement means replay routes remain closer to the evolving router under repeated updates. PR2 consistently improves this agreement, matching the intended behavior of predictive replay.

RQ3(b): PR2 reduces replay divergence. Figure 6 (right) shows a per-layer divergence proxy between replay and current routing. We use a coverage-gap proxy $1 - \sum_{j \in I_{\text{replay},t}^{(l)}} \rho_{\theta,t,j}^{(l)}$, i.e., the probability mass that the current router assigns *outside* the replayed top- k set. Lower values imply better approximation of online routing and therefore smaller replay-induced mismatch. The reduction trend matches the bias-control view in Eq. (14).

Top- k deviation distribution. Average agreement can hide rare severe route flips. We therefore inspect the full distribution of changed expert counts in top- k . For layer l and token t , the deviation count is $k - |I_{\text{replay},t}^{(l)} \cap I_{\text{cur},t}^{(l)}|$. PR2 shifts probability mass toward smaller deviations, indicating fewer large route mismatches.

Prediction horizon within one rollout batch. PR2 is designed for short-horizon evolution rather than arbitrarily long forecasting. Figure 8 plots agreement versus within-batch mini-step index $i \in \{1, \dots, \kappa\}$. Agreement decays as i grows, which is expected because routes become more

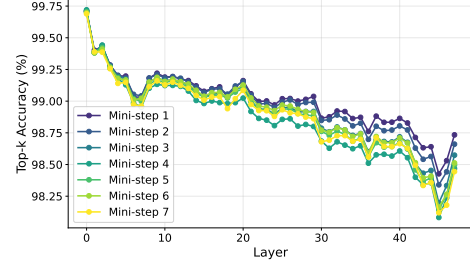


Figure 8. Prediction-horizon analysis. Top- k agreement between rollout-time predicted routes and routes at within-batch update step i . Agreement decreases with horizon but remains informative over multiple mini-steps.

off-policy later in the batch. Importantly, the decay is gradual, showing that predictor signals remain useful through multiple updates.

6. Conclusion

This work studies reinforcement learning for MoE-based LLMs in disaggregated pipelines and identifies router drift as a key source of instability. We interpret routing replay as replacing the online routing distribution with a cached replay route, and relate routing mismatch to deviations between the online routing distribution and the replay measure (Eq. (14)). This perspective explains why frozen replay improves short-term stability but can accumulate staleness over repeated off-policy updates.

Motivated by this analysis, we proposed **Predictive Routing Replay (PR2)**. PR2 predicts short-horizon router evolution during rollout, caches the resulting predicted top- k expert sets, and deterministically replays these sets during training for route consistency. Since deterministic top- k replay is non-differentiable, PR2 trains the predictor using a KL alignment objective between the current routing distribution and a predicted routing distribution (Eq. (15)), which serves as a smooth proxy for reducing replay mismatch. Across off-policy settings, PR2 consistently improves over complete PPO-style baselines (GRPO/GSPO/R2), while yielding smoother optimization trajectories and better routing-alignment diagnostics. These results suggest that predictive replay is a practical way to trade off stability and adaptation in MoE RL without modifying the core policy objective.

7. Limitations

PR2 is designed for short-horizon prediction. When off-policy reuse becomes very strong or optimization dynamics become highly non-stationary, predictor quality can degrade and replay routes may again become stale. Our theoretical analysis also relies on a distributional relaxation (sampling routes from a predicted routing distribution) and layerwise factorization assumptions; it does not explicitly model deter-

ministic top- k set selection or cross-layer route dependence.

From a systems perspective, PR2 introduces additional buffers and predictor-specific hyperparameters (especially learning-rate scaling), which require tuning. To keep memory practical under long contexts, our implementation caches router features for predictor learning only on a subsampled set of token positions, which may discard some fine-grained temporal information. In addition, our evaluation centers on PPO-style RL and reasoning-focused benchmarks; broader domains (e.g., tool-use-heavy tasks, multi-turn interaction, and non-PPO objectives) remain to be explored. Future work can investigate longer-horizon predictors, uncertainty-aware replay selection, and tighter integration between routing prediction and policy optimization.

Impact Statement

This paper presents a technical contribution aimed at improving the stability and effectiveness of reinforcement learning for Mixture-of-Experts (MoE) large language models. The primary goal is to advance the reliability of training methods for large-scale machine learning systems, particularly in research and engineering settings. While improved training stability and expert utilization may enable more capable and efficient language models, these advances do not introduce fundamentally new ethical risks beyond those already associated with large language models and reinforcement learning. Potential societal impacts therefore largely align with existing considerations around the deployment of powerful language models, including responsible use, safety, and governance. We do not identify any unique or immediate ethical concerns arising specifically from this work.

References

- Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., Arbus, E., Arora, R. K., Bai, Y., Baker, B., Bao, H., et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., and Vechev, M. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- Chen, A., Li, A., Gong, B., Jiang, B., Fei, B., Yang, B., Shan, B., Yu, C., Wang, C., Zhu, C., et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.
- Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- Dai, D., Dong, L., Ma, S., Zheng, B., Sui, Z., Chang, B., and Wei, F. Stablemoe: Stable routing strategy for mixture of experts. *arXiv preprint arXiv:2204.08396*, 2022.
- Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In *International conference on machine learning*, pp. 1407–1416. PMLR, 2018.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

- Gao, C., Zheng, C., Chen, X.-H., Dang, K., Liu, S., Yu, B., Yang, A., Bai, S., Zhou, J., and Lin, J. Soft adaptive policy optimization. *arXiv preprint arXiv:2511.20347*, 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Huang, Q., An, Z., Zhuang, N., Tao, M., Zhang, C., Jin, Y., Xu, K., Chen, L., Huang, S., and Feng, Y. Harder task needs more experts: Dynamic routing in moe models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12883–12895, 2024.
- Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., Casas, D. d. l., Hanna, E. B., Bressand, F., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.
- Kim, J., Song, M., Shin, S., and Son, S. Defending moe llms against harmful fine-tuning via safety routing alignment. *arXiv preprint arXiv:2509.22745*, 2025.
- Kool, W., Maddison, C. J., and Mnih, A. Unbiased gradient estimation with balanced assignments for mixtures of experts. In *I (Still) Can't Believe It's Not Better! NeurIPS 2021 Workshop*, 2021. URL <https://openreview.net/forum?id=Hvfva71ltcj>.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Lewis, M., Bhosale, S., Dettmers, T., Goyal, N., and Zettlemoyer, L. Base layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning*, pp. 6265–6274. PMLR, 2021.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.
- Liu, L., Gao, J., and Chen, W. Sparse backpropagation for moe training, 2023. URL <https://arxiv.org/abs/2310.00811>.
- Liu, L., Kim, Y. J., Wang, S., Liang, C., Shen, Y., Cheng, H., Liu, X., Tanaka, M., Wu, X., Hu, W., Chaudhary, V., Lin, Z., Zhang, C., Xue, J., Awadalla, H., Gao, J., and Chen, W. Grin: Gradient-informed moe, 2024b. URL <https://arxiv.org/abs/2409.12136>.
- Liu, Z., Tang, L., Jin, L., Li, H., Ranjan, N., Fan, D., Rohatgi, S., Fan, R., Pangarkar, O., Wang, H., et al. K2-v2: A 360-open, reasoning-enhanced llm. *arXiv preprint arXiv:2512.06201*, 2025.
- Ma, W., Zhang, H., Zhao, L., Song, Y., Wang, Y., Sui, Z., and Luo, F. Stabilizing moe reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- Mathematical Association of America. Amc student programs, 2026. URL <https://maa.org/student-programs/amc/>. Accessed January 25, 2026.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pp. 1928–1937. PmLR, 2016.
- OpenAI. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36: 53728–53741, 2023.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.

- Team, G. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025a. URL <https://arxiv.org/abs/2507.06261>.
- Team, G. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Team, K., Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025a.
- Team, L., Li, A., Liu, B., Hu, B., Li, B., Zeng, B., Ye, B., Tang, C., Tian, C., Huang, C., et al. Every activation boosted: Scaling general reasoner to 1 trillion open language foundation. *arXiv preprint arXiv:2510.22115*, 2025b.
- Team, M. Minimax-m1: Scaling test-time compute efficiently with lightning attention, 2025b. URL <https://arxiv.org/abs/2506.13585>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yao, F., Liu, L., Zhang, D., Dong, C., Shang, J., and Gao, J. Your efficient rl framework secretly brings you off-policy rl training, August 2025. URL <https://fengyao.notion.site/off-policy-rl>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Zhang, D., Wu, X., Huang, S., Hao, Y., Dong, L., Chi, Z., Sui, Z., and Wei, F. Towards stable and effective reinforcement learning for mixture-of-experts. *arXiv preprint arXiv:2510.23027*, 2025.
- Zhang, Y. and Math-AI, T. American invitational mathematics examination (aime).
- Zhao, Y., Liu, Y., Liu, J., Chen, J., Wu, X., Hao, Y., Lv, T., Huang, S., Cui, L., Ye, Q., Wan, F., and Wei, F. Geometric-mean policy optimization. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=nCEs0tSwc2>.
- Zheng, C., Dang, K., Yu, B., Li, M., Jiang, H., Lin, J., Liu, Y., Lin, H., Wu, C., Hu, F., et al. Stabilizing reinforcement learning with llms: Formulation and practices. *arXiv preprint arXiv:2512.01374*, 2025a.
- Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025b.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- Zhu, T., Qu, X., Dong, D., Ruan, J., Tong, J., He, C., and Cheng, Y. Llama-moe: Building mixture-of-experts from llama with continual pre-training. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 15913–15923, 2024.
- Zhu, Z., Xie, C., Lv, X., and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.
- Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

A. PPO-style Reinforcement Learning

A.1. Notation and Setup

We consider an autoregressive policy π_θ generating completion $y = (y_1, \dots, y_T)$ for prompt x . At token t , the state/action are $s_t = (x, y_{<t})$ and $a_t = y_t$. Rollouts are sampled from $\mu_{\theta_{\text{old}}}$, while optimization updates π_θ . We denote the training-engine old snapshot by $\pi_{\theta_{\text{old}}}$ and use $\text{Clip}(u, 1 - \epsilon, 1 + \epsilon)$ for PPO clipping (Schulman et al., 2017; Yu et al., 2025).

A.2. Proximal Policy Optimization (PPO)

PPO optimizes a clipped surrogate objective with token-level ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (17)$$

Given advantage A_t ,

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E} \left[\sum_{t=1}^T \min(r_t(\theta)A_t, \text{Clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t) \right]. \quad (18)$$

A KL regularizer to reference policy π_{ref} is often added:

$$\mathcal{L}_{\text{PPO+KL}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) - \beta \mathbb{E} \left[\sum_{t=1}^T D_{\text{KL}}(\pi_\theta(\cdot | s_t) || \pi_{\text{ref}}(\cdot | s_t)) \right]. \quad (19)$$

A.3. Group Relative Policy Optimization (GRPO)

GRPO uses group-relative advantages without value-function training (Shao et al., 2024; Guo et al., 2025). Given a group of G rollouts $\{y^{(g)}\}_{g=1}^G$ for prompt x , GRPO defines

$$A_t^{(g)} = \frac{R(x, y^{(g)}) - \frac{1}{G} \sum_{g'=1}^G R(x, y^{(g')})}{\text{Std}(\{R(x, y^{(g')})\}_{g'=1}^G) + \epsilon_A}. \quad (20)$$

The GRPO objective follows the PPO clipping form with these group-relative advantages:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\sum_{g=1}^G \sum_{t=1}^T \min(r_t^{(g)}(\theta)A_t^{(g)}, \text{Clip}(r_t^{(g)}(\theta), 1 - \epsilon, 1 + \epsilon)A_t^{(g)}) \right]. \quad (21)$$

B. Routing Replay Gradient Bias Bound

We restate the TV-based gradient mismatch bound used in Section 4. Let $g_t^*(\theta)$ and $g_t(q_t)$ be defined as

$$\begin{aligned} g_t^*(\theta) &:= \mathbb{E}_{\mathcal{R}_t \sim \rho_{\theta,t}(\cdot | s_t)} [A_t \nabla_{\theta} \log \pi_{\theta}(y_t | s_t, \mathcal{R}_t)], \\ g_t(q_t) &:= \mathbb{E}_{\mathcal{R}_t \sim q_t(\cdot | s_t)} [A_t \nabla_{\theta} \log \pi_{\theta}(y_t | s_t, \mathcal{R}_t)]. \end{aligned}$$

By total variation duality,

$$\|g_t^*(\theta) - g_t(q_t)\| \leq 2M_t D_{\text{TV}}(\rho_{\theta,t}(\cdot | s_t), q_t(\cdot | s_t)), \quad (22)$$

with $M_t \geq \|A_t \nabla_{\theta} \log \pi_{\theta}(y_t | s_t, \mathcal{R}_t)\|$. Eq. (22) directly links replay mismatch and gradient bias.

C. Full Proof of Proposition 4.1

Proof. Applying Eq. (22) with $q_t = \hat{q}_t$ gives

$$\|g_t^*(\theta) - g_t(\hat{q}_t)\| \leq 2M_t D_{\text{TV}}(\rho_{\theta,t}, \hat{q}_t).$$

By Pinsker's inequality,

$$D_{\text{TV}}(\rho_{\theta,t}, \hat{q}_t) \leq \sqrt{\frac{1}{2} D_{\text{KL}}(\rho_{\theta,t} \parallel \hat{q}_t)}.$$

Hence

$$\|g_t^*(\theta) - g_t(\hat{q}_t)\| \leq 2M_t \sqrt{\frac{1}{2} D_{\text{KL}}(\rho_{\theta,t} \parallel \hat{q}_t)}.$$

Under layerwise factorization,

$$\rho_{\theta,t}(\mathcal{R}_t) = \prod_{l=1}^L \rho_{\theta,t}^{(l)}(\mathcal{I}_t^{(l)}), \quad \hat{q}_t(\mathcal{R}_t) = \prod_{l=1}^L \hat{q}_t^{(l)}(\mathcal{I}_t^{(l)}),$$

so

$$D_{\text{KL}}(\rho_{\theta,t} \parallel \hat{q}_t) = \sum_{l=1}^L D_{\text{KL}}(\rho_{\theta,t}^{(l)} \parallel \hat{q}_t^{(l)}).$$

Therefore,

$$\|g_t^*(\theta) - g_t(\hat{q}_t)\| \leq 2M_t \sqrt{\frac{1}{2} \sum_{l=1}^L D_{\text{KL}}(\rho_{\theta,t}^{(l)} \parallel \hat{q}_t^{(l)})}.$$

Taking expectation over t , using $M_t \leq M$, and Jensen's inequality,

$$\mathbb{E}_t[\|g_t^*(\theta) - g_t(\hat{q}_t)\|] \leq M \sqrt{2 \sum_{l=1}^L \mathbb{E}_t[D_{\text{KL}}(\rho_{\theta,t}^{(l)} \parallel \hat{q}_t^{(l)})]} = M \sqrt{2 \mathcal{L}_{\text{PR2}}^{(\text{post})}},$$

where the last equality uses Eq. (15). □

D. Pseudo Code of Predictive Routing Replay (PR2)

We provide rollout-phase and training-phase pseudocode in Algorithms 1 and 2. The structure follows standard replay implementations with explicit route/feature buffers.

Algorithm 1 Predictive Routing Replay (PR2) in $\mu_{\theta_{\text{old}}}$ phase (rollout / inference engine)

Require: Token position τ , hidden feature h_{old} , router weights W_{old} , bias predictor W_p , expert set $\{E_{\text{old}1}, E_{\text{old}2}, \dots, E_{\text{old}n}\}$, expert path buffer $\mathcal{P}$, feature buffer $\mathcal{F}$, and a subsampling rule $\text{ShouldCache}(\tau) \in \{0, 1\}$.

Ensure: MoE layer output o_{old} .

```

1: // Routing information
2: Calculate router logits  $p_{\text{old}} = h_{\text{old}} W_{\text{old}}$ .
3: Calculate logit bias  $b = h_{\text{old}} W_p$ .
4: Calculate predicted routing distribution  $\hat{q} = \text{Softmax}(p_{\text{old}} + b)$  and expert set  $\mathcal{I}_{\text{old}} = \text{TopK}(\hat{q}, k)$ .
5: // Record to buffers
6: Update expert path buffer  $\text{Push}(\mathcal{P}, \mathcal{I}_{\text{old}})$ .
7: if  $\text{ShouldCache}(\tau) = 1$  then
8:   Update feature buffer  $\text{Push}(\mathcal{F}, (\tau, h_{\text{old}}, p_{\text{old}}))$ .
9: end if
10: // Model output
11: Calculate output  $o_{\text{old}} = \sum_{j \in \mathcal{I}_{\text{old}}} \hat{q}_j E_{\text{old},j}(h_{\text{old}})$ .
    
```

Algorithm 2 Predictive Routing Replay (PR2) in π_{θ} phase (training)

Require: Mini-step index $i \in \{1, \dots, \kappa\}$, token position τ , hidden feature h , router weights W , bias predictor W_p , expert set $\{E_1, E_2, \dots, E_n\}$, expert path buffer $\mathcal{P}$, feature buffer $\mathcal{F}$ (keyed by τ).

Ensure: MoE layer output o , predictive loss $\mathcal{L}_{\text{PR2}}$.

```

1: // Routing information
2: Calculate router logits  $p = hW$ .
3: Calculate current routing distribution  $\rho = \text{Softmax}(p)$ .
4: Read replayed expert set from buffer  $\mathcal{I} \leftarrow \text{Pop}(\mathcal{P})$ .
5: // Model output
6: Calculate output  $o = \sum_{j \in \mathcal{I}} \rho_j E_j(h)$ .
7: // Predictive loss (evaluated on cached feature subset)
8: if  $i == 1$  then
9:   Set predictive loss  $\mathcal{L}_{\text{PR2}} = 0$ .
10: else
11:   if  $(\tau, h_{\text{old}}, p_{\text{old}}) \in \mathcal{F}$  then
12:     Read cached features  $(h_{\text{old}}, p_{\text{old}}) \leftarrow \text{Query}(\mathcal{F}, \tau)$ .
13:     Recalculate logit bias  $b = h_{\text{old}} W_p$ .
14:     Calculate predicted distribution  $\hat{q} = \text{Softmax}(p_{\text{old}} + b)$ .
15:     Set predictive loss (post-routing)  $\mathcal{L}_{\text{PR2}} = D_{\text{KL}}(\langle \rho \rangle \| \hat{q})$ .
16:   else
17:     Set predictive loss  $\mathcal{L}_{\text{PR2}} = 0$ .
18:   end if
19: end if
    
```

E. Implementation Details

PR2 off2: post **2e6** and 1e4, delta 1e4 PR2 off4: post , delta **1e3** PR2 off8: post **1e4**, delta 1e3

Qwen3-MoE-30B-A3B Base. During rollout we use $\mathcal{B}_{\text{global}} = 64$ and sample 8 responses per prompt with maximum length 16K tokens. We sweep $\mathcal{B}_{\text{update}} \in \{32, 16, 8\}$ for off-2/off-4/off-8, with learning rates $\{2 \times 10^{-6}, 1.5 \times 10^{-6}, 1 \times 10^{-6}\}$. Clip ratios are $\epsilon_{\text{low}} = 0.2$ and $\epsilon_{\text{high}} = 0.28$. Each run uses 32 NVIDIA H200 GPUs and lasts about 72 hours.

R3 reproduction protocol. R3 uses the same data, optimizer family, off-policy settings, and validation schedule as PR2 for fair comparison. Routes are replayed from rollout-time routing decisions in both log-prob and gradient phases.

Table 3. Model configuration used in all main experiments.

Model	Layers	Heads (Q/KV)	# Experts (Total/Activated)	Context Length
Qwen3-30B-A3B	48	32 / 4	128 / 8	128K

Predictive Routing Replay. PR2 adds a lightweight predictor head to each router. During rollout, predicted routes are generated by $\hat{q} = \text{Softmax}(p_{\text{old}} + b)$ and the resulting top- k expert sets are cached in route buffers for deterministic replay. To keep memory practical under long contexts, router features used for predictor learning (e.g., $(h_{\text{old}}, p_{\text{old}})$) are cached only on a subsampled set of token positions; the post-routing matching loss is evaluated on this subset. Predictors are zero-initialized and trained with a separate learning-rate multiplier.